IO_PWD.h File Reference

IO Driver functions for timer input channels. More...

```
#include "IO_Driver.h"
```

Data Structures

struct	**IO_PWD_INC_SAFETY_CONF** Safety configuration for the Incremental or Counter PWD inputs. More...
struct	**IO_PWD_PULSE_SAMPLES** PWD pulse-width data structure. More...
struct	**IO_PWD_CPLX_SAFETY_CONF** Safety configuration for the Complex PWD inputs. More...
struct	**IO_PWD_CPLX_CONF** Complex configuration for the Universal PWD inputs. More...
struct	**IO_PWD_CNT_CONF** Edge counter configuration for the Universal PWD inputs. More...
struct	**IO_PWD_INC_CONF** Incremental configuration for the Universal PWD inputs. More...
struct	**IO_PWD_UNIVERSAL_SAFETY_CONF** Safety configuration for the Universal PWD inputs. More...

Macros

High/low time

Specifies whether the high,low or both(period) time shall be captured

#define	**IO_PWD_LOW_TIME** 0U
#define	**IO_PWD_HIGH_TIME** 1U
#define	**IO_PWD_PERIOD_TIME** 2U

Variable edge

Specify the variable edge of the input signal.
If the rising edge is variable, the frequency is measured between the surrounding falling edges.

#define	**IO_PWD_RISING_VAR** 2U
#define	**IO_PWD_FALLING_VAR** 3U

Pull up / Pull down configuration for timer inputs

Configuration of the pull up or pull down resistors on the timer inputs. These defines can be used for the `pupd` parameter of the functions **`IO_PWD_ComplexInit()`**, **`IO_PWD_IncInit()`** and **`IO_PWD_CountInit()`**.

#define **IO_PWD_NO_PULL** 0x03U

#define **IO_PWD_PU_10K** 0x01U

#define **IO_PWD_PD_10K** 0x00U

#define **IO_PWD_PD_90** 0x02U

PWD maximum pulse pulse-width samples

Maximum pulse-width samples that can be stored in the data structure **`IO_PWD_PULSE_SAMPLES`**

#define **IO_PWD_MAX_PULSE_SAMPLES** 8U

Counting mode

Specify the counting mode of a incremental interface.

#define **IO_PWD_INC_2_COUNT** 0x03U

#define **IO_PWD_INC_1_COUNT** 0x01U

edge count

Specify on which edge shall be counted

#define **IO_PWD_RISING_COUNT** 1U

#define **IO_PWD_FALLING_COUNT** 2U

#define **IO_PWD_BOTH_COUNT** 3U

count direction

Specify the counting direction

#define **IO_PWD_UP_COUNT** 0U

#define **IO_PWD_DOWN_COUNT** 1U

Functions

IO_ErrorType **IO_PWD_ComplexInit** (**ubyte1** timer_channel, **ubyte1** pulse_mode, **ubyte1** freq_mode, **ubyte1** capture_count, **ubyte1** pupd, **IO_PWD_CPLX_SAFETY_CONF** const *const safety_conf)

Setup single timer channel that measures frequency and pulse-width at the same time. More...

IO_ErrorType **IO_PWD_ComplexGet** (**ubyte1** timer_channel, **ubyte4** *const frequency, **ubyte4** *const pulse_width, **bool** *const pin_value, **IO_PWD_PULSE_SAMPLES** *const pulse_samples)

	Get the frequency and the pulse-width from the specified timer channel. More...
IO_ErrorType	**IO_PWD_ComplexDeInit** (**ubyte1** timer_channel) Deinitializes a complex PWD input. More...
IO_ErrorType	**IO_PWD_IncInit** (**ubyte1** inc_channel, **ubyte1** mode, **ubyte2** count_init, **ubyte1** pupd, **IO_PWD_INC_SAFETY_CONF** const *const safety_conf) Setup a single incremental interface. More...
IO_ErrorType	**IO_PWD_IncGet** (**ubyte1** inc_channel, **ubyte2** *const count, **bool** *const pin_value_0, **bool** *const pin_value_1) Get the counter value of an incremental interface. More...
IO_ErrorType	**IO_PWD_IncSet** (**ubyte1** inc_channel, **ubyte2** count) Set the counter value of an incremental interface. More...
IO_ErrorType	**IO_PWD_IncDeInit** (**ubyte1** inc_channel) Deinitialize an incremental interface. More...
IO_ErrorType	**IO_PWD_CountInit** (**ubyte1** count_channel, **ubyte1** mode, **ubyte1** direction, **ubyte2** count_init, **ubyte1** pupd, **IO_PWD_INC_SAFETY_CONF** const *const safety_conf) Setup a single counter channel. More...
IO_ErrorType	**IO_PWD_CountGet** (**ubyte1** count_channel, **ubyte2** *const count, **bool** *const pin_value) Get the counter value of a counter channel. More...
IO_ErrorType	**IO_PWD_CountSet** (**ubyte1** count_channel, **ubyte2** count) Set the counter value of a counter channel. More...
IO_ErrorType	**IO_PWD_CountDeInit** (**ubyte1** count_channel) Deinitialize a single counter channel. More...
IO_ErrorType	**IO_PWD_UniversalInit** (**ubyte1** timer_channel, **IO_PWD_CPLX_CONF** const *const cplx_conf, **IO_PWD_CNT_CONF** const *const cnt_conf, **IO_PWD_INC_CONF** const *const inc_conf, **ubyte1** pupd, **IO_PWD_UNIVERSAL_SAFETY_CONF** const *const safety_conf) Setup a single universal timer channel. More...
IO_ErrorType	**IO_PWD_UniversalGet** (**ubyte1** timer_channel, **ubyte4** *const frequency, **ubyte4** *const pulse_width, **IO_PWD_PULSE_SAMPLES** *const pulse_samples, **ubyte2** *const edge_count, **ubyte2** *const inc_count, **bool** *const primary_pin_value, **bool** *const secondary_pin_value) Get the measurement results from the specified universal timer channel. More...
IO_ErrorType	**IO_PWD_UniversalSet** (**ubyte1** timer_channel, **ubyte2** const *const edge_count, **ubyte2** const *const inc_count) Set the counter values of an universal timer channel. More...
IO_ErrorType	**IO_PWD_UniversalDeInit** (**ubyte1** timer_channel) Deinitialize a single universal timer channel. More...
IO_ErrorType	

	IO_PWD_GetVoltage (**ubyte1** pwd_channel, **ubyte2** *const voltage, **bool** *const fresh) Get the voltage feedback of a timer input channel. More...
IO_ErrorType	**IO_PWD_GetCurrent** (**ubyte1** pwd_channel, **ubyte2** *const current, **bool** *const fresh) Get the current feedback of a timer input channel. More...
IO_ErrorType	**IO_PWD_ResetProtection** (**ubyte1** pwd_channel, **ubyte1** *const reset_cnt) Reset the FET protection for a timer input channel. More...

Detailed Description

IO Driver functions for timer input channels.

Contains all service functions for the PWD (Pulse Width Demodulation). There are three groups of timer inputs available:

- `IO_PWD_00`..`IO_PWD_05`:
  - Complex mode: Can be configured to measure frequency and pulse-width at the same time. Additionally, a pull up/down interface can be configured. These inputs can accumulate up to 8 pulse samples. An average value is calculated automatically, but all samples are available on demand. These inputs support voltage and current signals (7mA/14mA). For current signals an additional range check is done.
  - Incremental mode: Can be configured to read incremental (relative) encoders. In this case two inputs are reserved for one incremental encoder (clock and direction) interface. Additionally, a pull up/down interface can be configured. The incremental interface will decrement when the 1st channel is leading and increment when the 2nd channel is leading. These inputs support voltage signals only.
  - Count mode: Can be configured to count rising, falling or both edges. Additionally, a pull up/down interface can be configured. These inputs support voltage signals only.
  - Universal mode: Can be configured as a combination of complex, incremental and count mode.
- `IO_PWD_06`..`IO_PWD_11`:
  - Complex mode: Can be configured to measure frequency and pulse-width at the same time. Additionally, a pull up/down interface can be configured. These inputs can accumulate up to 8 pulse samples. An average value is calculated automatically. The different pulse samples cannot be gathered. These inputs support voltage signals only.
  - Incremental mode: Can be configured to read incremental (relative) encoders. In this case two inputs are reserved for one incremental encoder (clock and direction) interface. Additionally, a pull up/down interface can be configured. The incremental interface will decrement when the 1st channel is leading and increment when the 2nd channel is leading. These inputs support voltage signals only.
  - Count mode: Can be configured to count rising, falling or both edges. Additionally, a pull up/down interface can be configured. These inputs support voltage signals only.

- **IO_PWD_12..IO_PWD_19:**
  - Complex mode: Can be configured to measure frequency and pulse-width at the same time. However, these inputs do not accumulate a number of samples. Instead every pulse sample has to be handled directly. These inputs support voltage signals only.

PWD-API Usage:

- **Examples for PWD API functions**

PWD input protection

Each PWD input that is configured for complex mode in combination with a current sensor is individually protected against overcurrent. Whenever two consecutive samples above 54326uA are measured, the input protection is enabled latest within 22ms.

When entering the protection state, the PWD input has to remain in this state for at least 1s. After this wait time the PWD input can be reenabled via function **IO_PWD_ResetProtection()**. Note that the number of reenabling operations for a single PWD input is limited to 10. Refer to function **IO_PWD_ResetProtection()** for more information on how to reenable a PWD input.

Attention

PWD input protection is only applicable to PWD channels **IO_PWD_00** .. **IO_PWD_05** when configured for complex mode in combination with current sensors.

PWD code examples

Examples for using the PWD API

PWD initialization example

```
// setup complex timer input (frequency and pulse measurement) for a voltage
       signal
IO_PWD_ComplexInit(IO_PWD_00,                // timer input channel
                   IO_PWD_HIGH_TIME,         // measure high time
                   IO_PWD_FALLING_VAR,       // select falling edge as
       variable
                   8,                        // set number of accumulations
                   IO_PWD_PU_10K,            // select pull up resistor
                   NULL);                    // not safety critical
```

PWD task example

```
ubyte4 freq_8_val;
ubyte4 pulse_8_val;
bool pin_value;

// read complex timer values (frequency and pulse value
IO_PWD_ComplexGet(IO_PWD_00,
                  &freq_8_val,
                  &pulse_8_val,
                  &pin_value,     // read digital pin value
                  NULL);          // pulse samples not needed
```

Macro Definition Documentation

#define IO_PWD_BOTH_COUNT 3U

count on both edges

#define IO_PWD_DOWN_COUNT 1U

count down

#define IO_PWD_FALLING_COUNT 2U

count on a falling edge

#define IO_PWD_FALLING_VAR 3U

falling edge of the input signal is the variable one

#define IO_PWD_HIGH_TIME 1U

capture the high time of the input signal

#define IO_PWD_INC_1_COUNT 0x01U

count on any edge of 1st input channel only:

- count on edges of `IO_PWD_00` for 1st incremental interface
- count on edges of `IO_PWD_02` for 2nd incremental interface
- count on edges of `IO_PWD_04` for 3rd incremental interface
- count on edges of `IO_PWD_06` for 4th incremental interface
- count on edges of `IO_PWD_08` for 5th incremental interface
- count on edges of `IO_PWD_10` for 6th incremental interface

#define IO_PWD_INC_2_COUNT 0x03U

count on any edge of the two input channels

#define IO_PWD_LOW_TIME 0U

capture the low time of the input signal

#define IO_PWD_MAX_PULSE_SAMPLES 8U

#define IO_PWD_NO_PULL 0x03U

fixed pull resistor

#define IO_PWD_PD_10K 0x00U

10 kOhm pull down

#define IO_PWD_PD_90 0x02U

90 Ohm pull down

#define IO_PWD_PERIOD_TIME 2U

capture the high and low time of the input signal

#define IO_PWD_PU_10K 0x01U

10 kOhm pull up

#define IO_PWD_RISING_COUNT 1U

count on a rising edge

#define IO_PWD_RISING_VAR 2U

rising edge of the input signal is the variable one

#define IO_PWD_UP_COUNT 0U

count up

Function Documentation

IO_ErrorType IO_PWD_ComplexDeInit (ubyte1 timer_channel)

Deinitializes a complex PWD input.

Parameters

timer_channel Timer channel:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`
- `IO_PWD_12` .. `IO_PWD_19`

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

```
IO_ErrorType IO_PWD_ComplexGet ( ubyte1                          timer_channel,
                                 ubyte4 *const                   frequency,
                                 ubyte4 *const                   pulse_width,
                                 bool *const                     pin_value,
                                 IO_PWD_PULSE_SAMPLES *const     pulse_samples
                               )
```

Get the frequency and the pulse-width from the specified timer channel.

Parameters

	timer_channel	Timer channel: • `IO_PWD_00` .. `IO_PWD_05` • `IO_PWD_06` .. `IO_PWD_11` • `IO_PWD_12` .. `IO_PWD_19`
`[out]`	**frequency**	Accumulated frequency in mHz (1/1000 Hz)
`[out]`	**pulse_width**	Accumulated pulse-width in us
`[out]`	**pin_value**	Digital value of the pin
`[out]`	**pulse_samples**	Contains each pulse-width measure sample

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_PWD_NOT_FINISHED	not enough edges to accumulate a result
IO_E_PWD_OVERFLOW	a timer overflow occurred
IO_E_PWD_CURRENT_THRESH_HIGH	the last measured current signal was above the threshold of 20 mA
IO_E_PWD_CURRENT_THRESH_LOW	the last measured current signal was below the threshold of 4 mA
IO_E_FET_PROT_ACTIVE	the input FET protection is active and has deactivated the internal switch. It can be reset with `IO_PWD_ResetProtection()` after the wait time is passed
IO_E_FET_PROT_REENABLE	the input FET protection is ready to be reset with `IO_PWD_ResetProtection()`
IO_E_FET_PROT_PERMANENT	

	the input FET is protected permanently because it was already reset more than 10 times
IO_E_NULL_POINTER	a NULL pointer has been passed to the function
IO_E_STARTUP	the input is in the startup phase
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Note

The return values **IO_E_PWD_CURRENT_THRESH_HIGH**, **IO_E_PWD_CURRENT_THRESH_LOW**, **IO_E_FET_PROT_ACTIVE**, **IO_E_FET_PROT_REENABLE** and **IO_E_FET_PROT_PERMANENT** will only be returned in case of a current sensor configuration (`pupd` == **IO_PWD_PD_90**)

Remarks

- The maximum frequency that can be measured with **IO_PWD_00** .. **IO_PWD_11** is 20kHz.
- The maximum frequency that can be measured with **IO_PWD_12** .. **IO_PWD_19** is 10kHz.
- The parameter `pin_value` is optional. If not needed, the parameter can be set **NULL** to ignore them.
- If each individual measured pulse-width sample is not needed, the parameter `pulse_samples` should be set to **NULL**

```
IO_ErrorType
IO_PWD_ComplexInit      ( ubyte1                                timer_channel,
                          ubyte1                                pulse_mode,
                          ubyte1                                freq_mode,
                          ubyte1                                capture_count,
                          ubyte1                                pupd,
                          IO_PWD_CPLX_SAFETY_CONF const *const  safety_conf
                        )
```

Setup single timer channel that measures frequency and pulse-width at the same time.

Parameters

timer_channel Timer channel:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`
- `IO_PWD_12` .. `IO_PWD_19`

pulse_mode Specifies the pulse mode

- `IO_PWD_HIGH_TIME`: configuration to measure pulse-high-time
- `IO_PWD_LOW_TIME`: configuration to measure pulse-low-time
- `IO_PWD_PERIOD_TIME`: configuration to measure pulse-high and low-time (Period)

freq_mode Specifies the variable edge

- `IO_PWD_RISING_VAR`: rising edge is variable this means, frequency is measured on falling edges
- `IO_PWD_FALLING_VAR`: falling edge is variable this means, frequency is measured on rising edges

capture_count Number of frequency/pulse-width measurements that will be accumulated (1..8)

pupd Pull up/down interface:

- `IO_PWD_NO_PULL`: fixed pull resistor
- `IO_PWD_PU_10K`: Pull up 10 kOhm
- `IO_PWD_PD_10K`: Pull down 10 kOhm
- `IO_PWD_PD_90`: Pull down 90 Ohm (for 7mA/14mA sensors)

`[in]` **safety_conf** Relevant safety configurations for the checker modules

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_CHANNEL_BUSY	the channel is currently used by another function
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist or is not available on the used ECU variant
IO_E_CH_CAPABILITY	the given channel is not a PWD channel or the used ECU variant does not support this function
IO_E_INVALID_PARAMETER	invalid parameter has been passed
IO_E_DRIVER_NOT_INITIALIZED	the common driver init function has not been called before
IO_E_FPGA_NOT_INITIALIZED	the FPGA has not been initialized
IO_E_INVALID_SAFETY_CONFIG	an invalid safety configuration has been passed
IO_E_DRV_SAFETY_CONF_NOT_CONFIG	global safety configuration is missing
IO_E_DRV_SAFETY_CYCLE_RUNNING	the init function was called after the task begin function
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Remarks

- The supported features depend on the selected channel:
  - **IO_PWD_00** .. **IO_PWD_05**:
    - `pulse_mode`: **IO_PWD_HIGH_TIME**, **IO_PWD_LOW_TIME** or **IO_PWD_PERIOD_TIME**
    - `freq_mode`: **IO_PWD_RISING_VAR** or **IO_PWD_FALLING_VAR**
    - `capture_count`: 1..8
    - `pupd`: **IO_PWD_PU_10K**, **IO_PWD_PD_10K** or **IO_PWD_PD_90**
    - `safety_conf`: Safety configuration
  - **IO_PWD_06** .. **IO_PWD_11**:
    - `pulse_mode`: **IO_PWD_HIGH_TIME**, **IO_PWD_LOW_TIME** or **IO_PWD_PERIOD_TIME**
    - `freq_mode`: **IO_PWD_RISING_VAR** or **IO_PWD_FALLING_VAR**
    - `capture_count`: 1..8
    - `pupd`: **IO_PWD_PU_10K** or **IO_PWD_PD_10K**

- **IO_PWD_12** .. **IO_PWD_19**:
  - pulse_mode: **IO_PWD_HIGH_TIME**, **IO_PWD_LOW_TIME** or **IO_PWD_PERIOD_TIME**
  - freq_mode: **IO_PWD_RISING_VAR** or **IO_PWD_FALLING_VAR**

Attention

Passing **IO_PWD_PD_90** as pupd parameter configures the input for a current sensor (7mA/14mA). In this case an additional range check on the current signal will be performed.

Remarks

- A channel that is initialized with this function can retrieve frequency and duty cycle by calling the function: **IO_PWD_ComplexGet()**
- The timing measurement for channels **IO_PWD_00** .. **IO_PWD_05** is based on a 27bit timer with a resolution of 0.5 us, therefore the period that shall be measured must be smaller than 67.108.863 us (~ 67 s).
- The timing measurement for channels **IO_PWD_06** .. **IO_PWD_11** is hardware limited to 10.000.000 us (= 10 s) with a resolution of 0.5 us. Because those timers work in an accumulating fashion the product of the measured period and the capture_count must be below this time.
- The timing measurement for channels **IO_PWD_12** .. **IO_PWD_19** is hardware limited to 10.000.000 us (= 10 s) with a resolution of 1 us.
- The maximum frequency that can be measured with **IO_PWD_00** .. **IO_PWD_11** is 20kHz.
- The maximum frequency that can be measured with **IO_PWD_12** .. **IO_PWD_19** is 10kHz.
- The driver captures exactly as many measurements are given in the parameter capture_count, except for **IO_PWD_12** .. **IO_PWD_19**.

Note

Until not all configured samples are captured, the driver doesn't return a value.

IO_ErrorType IO_PWD_CountDeInit (ubyte1 count_channel)

Deinitialize a single counter channel.

Parameters

count_channel Counter channel:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

```
IO_ErrorType IO_PWD_CountGet ( ubyte1          count_channel,
                               ubyte2 *const   count,
                               bool *const     pin_value
                             )
```

Get the counter value of a counter channel.

Parameters

count_channel Counter channel:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

`[out]` **count** Value of the counter (0..65535)

`[out]` **pin_value** Digital value of the pin

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_NULL_POINTER	a NULL pointer has been passed
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Remarks

The parameter `pin_value` is optional.

If not needed, the parameter can be set `NULL` to ignore them.

```
IO_ErrorType
IO_PWD_CountInit          ( ubyte1                              count_channel,
                            ubyte1                              mode,
                            ubyte1                              direction,
                            ubyte2                              count_init,
                            ubyte1                              pupd,
                            IO_PWD_INC_SAFETY_CONF const *const safety_conf
                          )
```

Setup a single counter channel.

Parameters

count_channel Counter channel:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

mode Specify on which edge shall be count

- `IO_PWD_RISING_COUNT`: count on a rising edge
- `IO_PWD_FALLING_COUNT`: count on a falling edge
- `IO_PWD_BOTH_COUNT`: count on a both edges

direction Specify the counting direction

- `IO_PWD_UP_COUNT`: counts up
- `IO_PWD_DOWN_COUNT`: counts down

count_init Init value of the counter (0..65535)

pupd Pull up/down interface:

- `IO_PWD_PU_10K`: Pull up 10 kOhm
- `IO_PWD_PD_10K`: Pull down 10 kOhm

`[in]` **safety_conf** Relevant safety configurations for the checker modules

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_CHANNEL_BUSY	the channel is currently used by another function
IO_E_INVALID_CHANNEL_ID	

	the given channel id does not exist or is not available on the used ECU variant
IO_E_CH_CAPABILITY	the given channel is not a PWD channel or the used ECU variant does not support this function
IO_E_INVALID_PARAMETER	invalid parameter has been passed
IO_E_DRIVER_NOT_INITIALIZED	the common driver init function has not been called before
IO_E_FPGA_NOT_INITIALIZED	the FPGA has not been initialized
IO_E_INVALID_SAFETY_CONFIG	an invalid safety configuration has been passed
IO_E_DRV_SAFETY_CONF_NOT_CONFIG	global safety configuration is missing
IO_E_DRV_SAFETY_CYCLE_RUNNING	the init function was called after the task begin function
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Remarks

- The parameter `safety_conf` is only supported for the PWD channels **IO_PWD_00** .. **IO_PWD_05**

```
IO_ErrorType IO_PWD_CountSet ( ubyte1  count_channel,
                               ubyte2  count
                             )
```

Set the counter value of a counter channel.

Parameters

count_channel Counter channel:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

count Value of the counter (0..65535)

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

```
IO_ErrorType IO_PWD_GetCurrent ( ubyte1          pwd_channel,
                                 ubyte2 *const   current,
                                 bool *const     fresh
                               )
```

Get the current feedback of a timer input channel.

Parameters

pwd_channel PWD channel:

- `IO_PWD_00` .. `IO_PWD_05`

`[out]` **current** Measured current in uA. Range: 0..20000 (0mA..20.000mA)

`[out]` **fresh** Indicates if a new value is available since the last call.

- `TRUE`: Value in "current" is valid
- `FALSE`: No new value available.

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_NULL_POINTER	a NULL pointer has been passed to the function
IO_E_PWD_CURRENT_THRESH_HIGH	the last measured current signal was above the threshold of 20 mA
IO_E_PWD_CURRENT_THRESH_LOW	the last measured current signal was below the threshold of 4 mA
IO_E_FET_PROT_ACTIVE	the input FET protection is active and has deactivated the internal switch. It can be reset with `IO_PWD_ResetProtection()` after the wait time is passed
IO_E_FET_PROT_REENABLE	the input FET protection is ready to be reset with `IO_PWD_ResetProtection()`
IO_E_FET_PROT_PERMANENT	the input FET is protected permanently because it was already reset more than 10 times
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CH_CAPABILITY	the given channel is not a PWD channel
IO_E_STARTUP	the input is in the startup phase
IO_E_CHANNEL_BUSY	

	the channel is currently used by another function
IO_E_REFERENCE	the internal reference voltage is out of range
IO_E_UNKNOWN	an unknown error occurred

Note

The return value **IO_E_CHANNEL_BUSY** will only be returned in case of a voltage sensor configuration (`pupd` != **IO_PWD_PD_90**)

Remarks

If there is no new current value available (for example the function **IO_PWD_GetCurrent()** gets called more frequently than the AD sampling) the flag `fresh` will be set to **FALSE**.

```
IO_ErrorType IO_PWD_GetVoltage ( ubyte1          pwd_channel,
                                 ubyte2 *const   voltage,
                                 bool *const     fresh
                               )
```

Get the voltage feedback of a timer input channel.

Parameters

pwd_channel PWD channel:

- IO_PWD_00 .. IO_PWD_05
- IO_PWD_06 .. IO_PWD_11

[out] **voltage** Measured voltage in mV. Range: 0..32000 (0V..32.000V)

[out] **fresh** Indicates if a new value is available since the last call.

- TRUE: Value in "voltage" is valid
- FALSE: No new value available.

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_NULL_POINTER	a NULL pointer has been passed to the function
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CH_CAPABILITY	the given channel is not a PWD channel
IO_E_STARTUP	the input is in the startup phase
IO_E_CHANNEL_BUSY	the channel is currently used by another function
IO_E_REFERENCE	the internal reference voltage is out of range
IO_E_UNKNOWN	an unknown error occurred

Note

The return value IO_E_CHANNEL_BUSY will only be returned in case of a current sensor configuration (pupd == IO_PWD_PD_90)

Remarks

If there is no new voltage value available (for example the function IO_PWD_GetVoltage() gets called more frequently than the AD sampling) the flag fresh will be set to FALSE.

IO_ErrorType IO_PWD_IncDeInit (ubyte1 inc_channel)

Deinitialize an incremental interface.

Parameters

inc_channel Channel of the incremental interface:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Note

- The function can be called with either the 1st or the 2nd channel of the incremental interface.

```
IO_ErrorType IO_PWD_IncGet ( ubyte1          inc_channel,
                             ubyte2 *const   count,
                             bool *const     pin_value_0,
                             bool *const     pin_value_1
                           )
```

Get the counter value of an incremental interface.

Parameters

inc_channel Channel of the incremental interface:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

`[out]` **count** Value of the incremental counter (0..65535)

`[out]` **pin_value_0** Digital value of pin 0 (1st channel)

`[out]` **pin_value_1** Digital value of pin 1 (2nd channel)

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_NULL_POINTER	a NULL pointer has been passed
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Note

- The function can be called with either the 1st or the 2nd channel of the incremental interface.
- The incremental interface will decrement when the 1st channel is leading and increment when the 2nd channel is leading.

Remarks

- The parameters `pin_value_0` and `pin_value_1` are optional. If not needed, these parameters can be set `NULL` to ignore them.

```
IO_ErrorType IO_PWD_IncInit ( ubyte1                              inc_channel,
                              ubyte1                              mode,
                              ubyte2                              count_init,
                              ubyte1                              pupd,
                              IO_PWD_INC_SAFETY_CONF const *const safety_conf
                            )
```

Setup a single incremental interface.

Parameters

inc_channel Channel of the incremental interface:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

mode Defines the counter behavior

- `IO_PWD_INC_2_COUNT`: Counts up/down on any edge of the two input channels
- `IO_PWD_INC_1_COUNT`: Counts up/down on any edge of the 1st input channel only:
  - count on `IO_PWD_00` for 1st incremental interface
  - count on `IO_PWD_02` for 2nd incremental interface
  - count on `IO_PWD_04` for 3rd incremental interface
  - count on `IO_PWD_06` for 4th incremental interface
  - count on `IO_PWD_08` for 5th incremental interface
  - count on `IO_PWD_10` for 6th incremental interface

count_init Init value of the incremental counter (0..65535)

pupd Pull up/down interface:

- `IO_PWD_PU_10K`: Pull up 10 kOhm
- `IO_PWD_PD_10K`: Pull down 10 kOhm

`[in]` **safety_conf** Relevant safety configurations for the checker modules

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist or is not available on the used ECU variant
IO_E_CH_CAPABILITY	

	the given channel is not capable for an incremental interface or the used ECU variant does not support this function
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CHANNEL_BUSY	the channel is currently used by another function
IO_E_INVALID_PARAMETER	invalid parameter has been passed
IO_E_DRIVER_NOT_INITIALIZED	the common driver init function has not been called before
IO_E_FPGA_NOT_INITIALIZED	the FPGA has not been initialized
IO_E_INVALID_SAFETY_CONFIG	an invalid safety configuration has been passed
IO_E_DRV_SAFETY_CONF_NOT_CONFIG	global safety configuration is missing
IO_E_DRV_SAFETY_CYCLE_RUNNING	the init function was called after the task begin function
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Remarks

- The parameter `safety_conf` is only supported for the PWD channels **IO_PWD_00** .. **IO_PWD_05**
- A channel that is initialized with this function can retrieve the counter value: **IO_PWD_IncGet()**
- Two PWD channels are used (needed) for one incremental interface.
  The pairs are defined in the following order:
  - **IO_PWD_00** and **IO_PWD_01** define the 1st incremental interface.
  - **IO_PWD_02** and **IO_PWD_03** define the 2nd incremental interface.
  - **IO_PWD_04** and **IO_PWD_05** define the 3rd incremental interface.
  - **IO_PWD_06** and **IO_PWD_07** define the 4th incremental interface.
  - **IO_PWD_08** and **IO_PWD_09** define the 5th incremental interface.
  - **IO_PWD_10** and **IO_PWD_11** define the 6th incremental interface.

Note

- The function initializes both PWD channels of the incremental interface, independently if the function gets called with the 1st or 2nd channel belonging to an incremental interface.
- The incremental interface will decrement when the 1st channel is leading and increment when the 2nd channel is leading.

```
IO_ErrorType IO_PWD_IncSet ( ubyte1  inc_channel,
                             ubyte2  count
                           )
```

Set the counter value of an incremental interface.

Parameters

inc_channel Channel of the incremental interface:

- `IO_PWD_00` .. `IO_PWD_05`
- `IO_PWD_06` .. `IO_PWD_11`

count Value of the incremental counter (0..65535)

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Note

- The function can be called with either the 1st or the 2nd channel of the incremental interface.

IO_ErrorType IO_PWD_ResetProtection (ubyte1 pwd_channel,
ubyte1 *const reset_cnt
)

Reset the FET protection for a timer input channel.

Parameters

pwd_channel PWD channel:

- `IO_PWD_00` .. `IO_PWD_05`

`[out]` **reset_cnt** Protection reset counter. Indicates how often the application already reset the protection.

Returns

IO_ErrorType

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_FET_PROT_NOT_ACTIVE	no input FET protection is active
IO_E_FET_PROT_PERMANENT	the input FET is protected permanently because it was already reset more than 10 times
IO_E_FET_PROT_WAIT	the input FET protection can not be reset, as the wait time of 1s is not already passed
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_CH_CAPABILITY	the given channel is not a PWD channel
IO_E_CHANNEL_BUSY	the channel is currently used by another function
IO_E_UNKNOWN	an unknown error occurred

Note

The return value `IO_E_CHANNEL_BUSY` will only be returned in case of a voltage sensor configuration (`pupd` != `IO_PWD_PD_90`)

Remarks

When re-enabling the channel after it has been in protection state, it will do a transition to the startup state. Therefore the functions `IO_PWD_ComplexGet()` and `IO_PWD_GetCurrent()` will return `IO_E_STARTUP` for a certain time.

IO_ErrorType IO_PWD_UniversalDeInit (ubyte1 timer_channel)

Deinitialize a single universal timer channel.

Parameters

timer_channel Timer channel:

- `IO_PWD_00` .. `IO_PWD_05`

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

```
IO_ErrorType
IO_PWD_UniversalGet          ( ubyte1                          timer_channel,
                               ubyte4 *const                   frequency,
                               ubyte4 *const                   pulse_width,
                               IO_PWD_PULSE_SAMPLES *const     pulse_samples,
                               ubyte2 *const                   edge_count,
                               ubyte2 *const                   inc_count,
                               bool *const                     primary_pin_value,
                               bool *const                     secondary_pin_value
                             )
```

Get the measurement results from the specified universal timer channel.

Parameters

	timer_channel	Timer channel: • `IO_PWD_00` .. `IO_PWD_05`
[out]	**frequency**	Accumulated frequency in mHz (1/1000 Hz)
[out]	**pulse_width**	Accumulated pulse-width in us
[out]	**pulse_samples**	Contains each pulse-width measure sample
[out]	**edge_count**	Value of the edge counter (0..65535)
[out]	**inc_count**	Value of the incremental counter (0..65535)
[out]	**primary_pin_value**	Digital value of the primary pin
[out]	**secondary_pin_value**	Digital value of the secondary pin

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_PWD_NOT_FINISHED	not enough edges to accumulate a result
IO_E_PWD_OVERFLOW	a timer overflow occurred
IO_E_PWD_CURRENT_THRESH_HIGH	the last measured current signal was above the threshold of 20 mA
IO_E_PWD_CURRENT_THRESH_LOW	the last measured current signal was below the threshold of 4 mA
IO_E_FET_PROT_ACTIVE	the input FET protection is active and has deactivated the internal switch. It can be reset

	with `IO_PWD_ResetProtection()` after the wait time is passed
IO_E_FET_PROT_REENABLE	the input FET protection is ready to be reset with `IO_PWD_ResetProtection()`
IO_E_FET_PROT_PERMANENT	the input FET is protected permanently because it was already reset more than 10 times
IO_E_NULL_POINTER	a NULL pointer has been passed to the function
IO_E_STARTUP	the input is in the startup phase
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Note

- The function can be called with either the primary or the secondary channel of the incremental interface.
- The return values `IO_E_PWD_CURRENT_THRESH_HIGH`, `IO_E_PWD_CURRENT_THRESH_LOW`, `IO_E_FET_PROT_ACTIVE`, `IO_E_FET_PROT_REENABLE` and `IO_E_FET_PROT_PERMANENT` will only be returned in case of a current sensor configuration (`pupd` == `IO_PWD_PD_90`)

Remarks

- The maximum frequency that can be measured with `IO_PWD_00` .. `IO_PWD_05` is 20kHz.
- The parameters `frequency`, `pulse_width`, `pulse_samples`, `edge_count`, `inc_count` `primary_pin_value` and `secondary_pin_value` are optional. If not needed, these parameters can be set to `NULL` to ignore them.
- If each individual measured pulse-width sample is not needed, the parameter `pulse_samples` should be set to `NULL`

```
IO_ErrorType
IO_PWD_UniversalInit  ( ubyte1                                  timer_channel,
                        IO_PWD_CPLX_CONF const *const           cplx_conf,
                        IO_PWD_CNT_CONF const *const            cnt_conf,
                        IO_PWD_INC_CONF const *const            inc_conf,
                        ubyte1                                  pupd,
                        IO_PWD_UNIVERSAL_SAFETY_CONF const *const safety_conf
                      )
```

Setup a single universal timer channel.

Parameters

timer_channel Timer channel:

- `IO_PWD_00` .. `IO_PWD_05`

[in] **cplx_conf** Complex configuration

[in] **cnt_conf** Edge counter configuration

[in] **inc_conf** Incremental counter configuration

pupd Pull up/down interface:

- `IO_PWD_PU_10K`: Pull up 10 kOhm
- `IO_PWD_PD_10K`: Pull down 10 kOhm
- `IO_PWD_PD_90`: Pull down 90 Ohm (for 7mA/14mA sensors)

[in] **safety_conf** Relevant safety configurations for the checker modules

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_CHANNEL_BUSY	the channel is currently used by another function
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist or is not available on the used ECU variant
IO_E_CH_CAPABILITY	the given channel is not a PWD channel or the used ECU variant does not support this function
IO_E_INVALID_PARAMETER	invalid parameter has been passed
IO_E_DRIVER_NOT_INITIALIZED	the common driver init function has not been called before

IO_E_INVALID_SAFETY_CONFIG	an invalid safety configuration has been passed
IO_E_DRV_SAFETY_CONF_NOT_CONFIG	global safety configuration is missing
IO_E_DRV_SAFETY_CYCLE_RUNNING	the init function was called after the task begin function
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Attention

Passing **IO_PWD_PD_90** as `pupd` parameter configures the input for a current sensor (7mA/14mA). In this case an additional range check on the current signal will be performed. Note that this setting is invalid if the input is also configured for edge and/or incremental counter mode.

If a channel is configured for incremental mode, both the primary and secondary channel are redundantly configured using the provided complex, edge counter and safety configuration. Thus, the application must periodically call **IO_PWD_UniversalGet()** for both channels if either the complex or edge counter mode is configured for safety.

Remarks

- The supported features depend on the selected channel:
  - **IO_PWD_00** .. **IO_PWD_05**:
    - `pulse_mode`: **IO_PWD_HIGH_TIME**, **IO_PWD_LOW_TIME** or **IO_PWD_PERIOD_TIME**
    - `freq_mode`: **IO_PWD_RISING_VAR** or **IO_PWD_FALLING_VAR**
    - `capture_count`: 1..8
    - `pupd`: **IO_PWD_PU_10K**, **IO_PWD_PD_10K** or **IO_PWD_PD_90**
    - `safety_conf`: Safety configuration
- Parameter `safety_conf` can be set to NULL in order to initialize the input as non-safety-critical. To configure the input as safety-critical, parameter `safety_conf` must provide exactly one valid (i.e., not NULL) safety configuration. Providing zero or multiple safety configurations or a safety configuration for a mode that is not configured will fail with return code **IO_E_INVALID_SAFETY_CONFIG**.
- A channel that is initialized with this function can retrieve its measurement results by calling the function: **IO_PWD_UniversalGet()**
- The timing measurement for channels **IO_PWD_00** .. **IO_PWD_05** is based on a 27bit timer with a resolution of 0.5 us, therefore the period that shall be measured must be smaller than 67.108.863 us (~ 67 s).
- The maximum frequency that can be measured with **IO_PWD_00** .. **IO_PWD_05** is 20kHz.

Note

- For complex mode, the driver doesn't return a value as long as not all configured samples are captured.

IO_ErrorType IO_PWD_UniversalSet (ubyte1 timer_channel,
ubyte2 const *const edge_count,
ubyte2 const *const inc_count
)

Set the counter values of an universal timer channel.

Parameters

timer_channel Timer channel:

- `IO_PWD_00` .. `IO_PWD_05`

[in] **edge_count** Value of the edge counter (0..65535)

[in] **inc_count** Value of the incremental counter (0..65535)

Returns

IO_ErrorType:

Return values

IO_E_OK	everything fine
IO_E_INVALID_CHANNEL_ID	the given channel id does not exist
IO_E_CHANNEL_NOT_CONFIGURED	the given channel is not configured
IO_E_INTERNAL_CSM	an internal error occurred
IO_E_UNKNOWN	an unknown error occurred

Note

- When configured for incremental mode, the function can be called with either the primary or the secondary channel.

Remarks

- The parameters `edge_count` and `inc_count` are optional. If not needed, these parameters can be set to `NULL` to ignore them.